

Estructura de Computadores

Memoria Laboratorio 2

Daniel Deblas Payo / 100522144 / 100522144@alumnos.uc3m.es

Xiya Ye / 100522159 / 100522159@alumnos.uc3m.es

Grado de Ingeniería Informática - Grupo 85

1 de diciembre de 2024



Índice

Ejercicio 1.....	3
Ejercicio 2.....	6
Diferencias.....	6
Ventajas e inconvenientes.....	6
Propuesta de mejora.....	7
Conclusión.....	8

Ejercicio 1

Instrucción	Diseño	Señales	Decisiones de diseño
$lx R_{R1} U32$	1. $MAR \leftarrow PC$ 2. $MBR \leftarrow MP[MAR]$ $PC \leftarrow PC + 4$ 3. $BR[R_{R1}] \leftarrow MBR$, salto a fetch	1. (T2, C0) 2. (Ta, R, BW=11, M1=1, C1, M2=1, C2) 3. (T1, SelC=10101, LC, A0=1, B=1, C=0)	Como esta instrucción se codifica en 64 bits, hemos tenido en cuenta que nwords es 2, y que debemos hacer un pseudofetch, sumando al PC, 4. Llevamos el contador de programa a MAR para preparar la lectura de la instrucción. Después se lee la instrucción desde la memoria principal y se carga en el registro MBR. Posteriormente se actualiza el valor de contador de programa. Por último se carga el valor en el registro R1.
call U32	1. $MAR \leftarrow PC$ 2. $MBR \leftarrow MP[MAR]$ 3. $RT1 \leftarrow MBR$, $PC \leftarrow PC + 4$ 4. $SP \leftarrow SP - 4$ 5. $MBR \leftarrow PC$ 6. $MAR \leftarrow SP$ 7. $MP[MAR] \leftarrow MBR$ 8. $PC \leftarrow RT1$, salto a fetch	1. (T2, C0) 2. (Ta, R, BW=11, M1, C1) 3. (T1, C4, M2=1, C2) 4. (SelA=10, MR=1, MB=10, SelCop=01011, MC, T6, SelC=10, LC) 5. (T2, C1) 6. (SelA=10, MR=1, T9, C0) 7. (W, Ta, Td, BW=11) 8. (T4, C2, A0=1, B=1, C=0)	Al igual que la primera instrucción, esta también se codifica en 64 bits, por lo que tenemos que hacer lo mismo, nwords igual a 2 y sumarle a PC, 4. Primero se carga el valor de U32 en el registro RT1. Después se actualiza el valor del puntero de pila en 4 bytes para posteriormente guardar la dirección de retorno. Por último, se lleva el valor del contador de programa a la dirección de RT1.
return	1. $MAR \leftarrow SP$ 2. $MBR \leftarrow MP[MAR]$ 3. $PC \leftarrow MBR$ 4. $SP \leftarrow SP + 4$, salto a fetch	1. (SelA=10, MR=1, T9, C0) 2. (Ta, R, BW=11, M1=1, C1) 3. (T1, M2=0, C2) 4. (SelA=10, MR=1, MA=0, MB=10, SelCop=01010, MC, T6, SelC=10, LC, A0=1, B=1, C=0)	Primero se coge la dirección de retorno de la pila, lo llevamos a MAR, para leer la dirección, y posteriormente llevarlo a PC. Después se actualiza el valor del puntero de pila, sumándole 4 a SP, pasando por ALU.
stop	1. $SP \leftarrow R0$ 2. $PC \leftarrow R0$, salto a fetch	1. (SelA=0, MR=1, T9, SelC=10, LC) 2. (SelA=0, MR=1, T9, M2=0, C2, A0=1, B=1, C=0)	Se guarda 0 en el contador de programa y en la pila para que el programa finalice.

Instrucción	Diseño	Señales	Decisiones de diseño
vfill R _{R1} R _{R2} U8	1. RT2 $\leftarrow$ SR 2. RT1 (i) $\leftarrow$ R0 3. SR $\leftarrow$ RR1 - RT1 4. beq SR 0 fin (=) 5. MAR $\leftarrow$ RR2 + RT1 6. MBR $\leftarrow$ U8 7. MAR[addr] $\leftarrow$ MBR 8. RT1 $\leftarrow$ RT1 (i) + 1 9. salto a loop2 10. RR2 $\leftarrow$ RR1 + RR2 11. RT3 $\leftarrow$ R0 12. RR1 $\leftarrow$ RT1 13. SR $\leftarrow$ RT2, salto a fetch	1. (T8, C5) 2. (SelA=0, MR=1, T9, C4) 3. loop1:(SelB=10101, MA=1, SelCop=1011, MC=1, SelP=11, M7=1, C7) 4. (A0=0, B=0, C=110, MADDR=fin1) 5. (SelB=10000, MA=1, SelCop=1010, MC=1, T6, C0) 6. (SE=1, SIZE=1000, T3, C1) 7. (Ta, Td, W, BW=00) 8. (MA=1, MB=11, SelCop=1010, MC=1, T6, C4) 9. (A0=0, B=1, C=0, MADDR=loop1) 10. fin1: (SelA=10101, SelB=10000, SelCop=1010, MC=1, T6, SelC=10000, LC=1) 11. (SelA=0, MR=1, T9, C4) 12. (T4, SelC=10101, LC=1) 13. (T5, C7, A0=1, B=1, C=0)	Como el registro SR no debemos actualizarla, la guardamos en RT2, porque más adelante la vamos a actualizar. Inicializamos el registros RT1 a 0, porque va a ser nuestro contador i para el bucle, elegimos el RT1 en vez de algún registro en el banco de registros para no alterar los valores que contienen. Para hacer el bucle, lo que hacemos es restarle el registro R_{R1} al contador i (RT1), y ese resultado lo llevamos a SR para compararlo a 0, y usando el multiplexor A, dependiendo del si es igual a 0 o no, salte al fin del bucle o siga con el siguiente ciclo. Una vez dentro del bucle, para hacer addr $\leftarrow$ BR[R_{R2}] + i, le damos a RB el valor de R_{R2} y lo sumamos en ALU, llevándolo directamente a MAR dejar preparado a la hora de hacer MEM[addr] $\leftarrow$ U8. Una vez hecho eso, tenemos que llevar U8 a MBR, para poder escribirlo en la dirección calculada previamente, una cosa que hemos tenido que tener en cuenta es que al estar trabajando con bytes y no palabras, entonces BW=00. Para finalizar el bucle, iteramos el contador i (RT1) y hacemos el salto al bucle, gracias a la ayuda del multiplexor A, haciendo un salto incondicional. Una vez fuera del bucle, le asignamos a R_{R2} el valor de la suma de R_{R1} y R_{R2}, a R_{R1} le asignamos 0 (R0), esto lo hacemos en dos ciclos diferentes porque R_{R1} se encuentra en IR y R0 no. Finalmente le devolvemos a SR el valor que tenía al empezar la instrucción, que lo habíamos guardado en RT2, y como no hemos sobreescrito en RT2 nos podemos asegurar de que ese es su valor inicial.

Instrucción	Diseño	Señales	Decisiones de diseño
vadd R_{R1} R_{R2} R_{R3}	1. $RT1 \leftarrow SR$, $RT3 \leftarrow RT1$ 2. $RT1(i) \leftarrow R0$ 3. $SR \leftarrow RR1 - RT1$ 4. beq SR 0 fin (=) 5. $MAR \leftarrow RR2 + RT1$ 6. $MBR \leftarrow MP[addr]$ 7. $RT2 \leftarrow MBR$ 8. $MBR \leftarrow RT2 + RR3$ 9. $MP[MAR] \leftarrow MBR$ 10. $RT1 \leftarrow RT1(i) + 1$ 11. salto a loop2 12. $RR2 \leftarrow RR1 + RR2$ 13. $RT1 \leftarrow R0$ 14. $RR1 \leftarrow RT1$ 15. $SR \leftarrow RT3$, salto a fetch	1. (T8, SelC=110, MR=1, LC=1, MA=1, MB=11, SelCop=1100, MC=1, C6) 2. (SelA=0, MR=1, T9, C4) 3. loop2:(SelB=10101, MA=1, SelCop=1011, MC=1, SelP=11, M7=1, C7) 4. (A0=0, B=0, C=110, MADDR=fin2) 5. (SelB=10000, MA=1, SelCop=1010, MC=1, T6, C0) 6. (Ta, R, BW=00, M1=1, C1) 7. (T1, C5) 8. (SelA=1011, MB=01, SelCop=1010, MC=1, T6, C1) 9. (Ta, Td, W, BW=00) 10. (MA=1, MB=11, SelCop=1010, MC=1, T6, C4) 11. (A0=0, B=1, C=0, MADDR=loop2) 12. fin2: (SelA=10101, SelB=10000, SelCop=1010, MC=1, T6, SelC=10000, LC=1) 13. (SelA=0, MR=1, T9, C4) 14. (T4, SelC=10101, LC=1) 15. (T7, C7, A0=1, B=1, C=0)	En esta instrucción tenemos que guardar SR también, en este caso lo guardamos en RT3 (pasando por RT1 y posteriormente por ALU multiplicando por 1), porque RT1 y RT2 los vamos a estar usando. Como la condición del bucle de esta instrucción y el cálculo de addr es igual a la de vfill, hemos hecho lo mismo en esta. Para hacer $MP[addr] \leftarrow MP[addr] + R_{R3}$ leemos addr en la memoria principal, y lo llevamos a RT2, para poderlo sumar a R_{R3}, y finalmente llevar este resultado a MBR. Como en MAR ya está addr, porque lo pusimos en ciclos anteriores, simplemente escribimos el resultado en la memoria principal, teniendo en cuenta que trabajamos con bytes y no palabras. Después, iteramos nuestro contador i (R5) de la misma manera que en la instrucción vfill. Fuera del bucle, hacemos lo mismo que en la instrucción anterior, que se trata de asignarle a R_{R2}, la suma de R_{R1} y R_{R2}, a R_{R1} el valor 0 (R0), devolverle a SR el valor que tenía en el comienzo de la instrucción que lo tenemos en RT3, finalmente el salto a fetch.

Ejercicio 2

Diferencias

Las principales diferencias entre los dos casos, son en que en el caso donde tenemos la extensión, codificamos las instrucciones necesarias para el programa principal, así cuando vayamos a trabajar en ensamblador, usamos aparte de las instrucciones que nos da RISC-V, tenemos nuestras propias instrucciones que hacen justo lo que necesitamos hacer, esto hace que la función quede de manera más compacta y legible.

Por otro lado, en el segundo caso, debemos adaptarnos a las instrucciones de RISC-V, para poder hacer lo que nos pide la empresa. Por lo que, a la hora de hacer el ensamblador, aparte de la función principal, debemos crear otras dos funciones más, una para vadd (que va añadiendo los colores) y otro que sería vfill (que hace que desaparezca los colores, sustituyendolo por el negro), por lo que esta implementación nos quedaría más largo.

Otra gran diferencia sería que la implementación con extensión requiere de 12479 ciclos de reloj, mientras que la otra necesita 53987, esto se debe a que con la extensión, al estar los bucles en una instrucción, se pueden ejecutar múltiples operaciones en paralelo. Cuando no tenemos la extensión, al tener los bucles en el ensamblador, no puede hacer lo mismo que en el otro caso.

Ventajas e inconvenientes

La principal ventaja del uso de la extensión son los ciclos de reloj, ya que teniendo la extensión, se ejecutan en muchos menos ciclos, esto nos lleva a una gran reducción de energía a la empresa y a un mejor rendimiento del programa. Esto se debe a que al tener los bucles (que son los que tardan más en ejecutarse, osea que necesitan más ciclos) los implementamos en instrucciones, así que a la hora de ejecutar el programa, estos bucles pueden estar operando de manera simultánea con otras operaciones.

Por otro lado, al tener la extensión, a la hora de hacer el código, éste será mucho menos complejo, siendo también mucho más compacto. No obstante, esto nos lleva también a la complejidad del diseño del hardware, requiriendo más tiempo y coste de fabricación del diseño del procesador por la nueva extensión. Asimismo, la implementación sin la extensión se puede adaptar a cualquier procesador RISC-V, mientras que con la extensión está limitada.

Propuesta de mejora

Como hemos podido comprobar, la versión donde incluye la extensión nueva, tiene muchas más ventajas que en el otro caso, por razones como el ahorro de energía a la empresa, la rapidez que este tiene o el hecho de que el código en ensamblador queda mucho más compacto y robusto.

Como podemos ver en la siguiente tabla, los ciclos de reloj en el caso de sin extensión es 5 veces más que con la extensión, teniendo aproximadamente 80,63% de mejora en comparación.

	Ciclos de reloj sin extensión	Ciclos de reloj con extensión	Mejora (%)
Semáforo 8*24	53987	10457	80,63%

Por lo que podemos decir que haciendo en vez de programar los bucles (que suelen ser los que más ciclos de reloj necesitan) en las instrucciones, puede que así el coste inicial para el hardware especializado pueda ser más elevado, pero éste se compensa con la energía que se ahorrará en el futuro para la empresa.

Es por esto, que también consideramos que si se va a trabajar con la pantalla, merece la pena disponer de esta nueva extensión, pues el tiempo en este caso es muy importante, con la extensión aparecerán los píxeles mucho más rápido.

Conclusión

La realización de este trabajo nos ha ayudado a entender bien los fundamentos de la microprogramación y cómo se relaciona con las instrucciones del lenguaje ensamblador. Por otro lado, también nos ha servido para hacernos a la idea de como funciona el procesador gracias a la simulación que hacía WepSIM.

La principal dificultad que hemos encontrado ha sido usar las instrucciones `vfill` y `vadd` debido a que hemos tenido algunos errores a la hora de usar el MUX A para poner las condiciones de los bucles, pero una vez que entendimos el funcionamiento de éste, pudimos hacer el bucle sin problema.

Un problema que hemos tenido en las instrucciones `vfill` y `vadd`, ha sido ver que no era necesario usar el banco de registros para preservar el registro SR y el contador i, sino que se podía guardar en los registros temporales. Además nos hemos dado cuenta de que incluso mejor, porque a la hora de guardarlo en algún registro del banco de registros, podríamos estar sobrescribiendo sobre algún valor, por lo que habría que preservarlo también.

Otra dificultad que nos hemos encontrado haciendo las instrucciones `vfill` y `vadd` ha sido darnos cuenta de que estábamos trabajando con bytes y no palabras, por lo que al guardar el valor en la dirección calculada, debemos tener en cuenta que se tratan de bytes. Con este mismo problema nos hemos encontrado a la hora de hacer las funciones `vfill` y `vadd` en el ensamblador, ya que no debíamos usar `lw` y `sw`, sino `lb` y `sb`.

Para realizar la práctica hemos dedicado aproximadamente 42 horas, principalmente hemos estado bastante tiempo entendiendo la práctica y posteriormente haciendo el primer ejercicio.